



Preparando a robustez da aplicação

Programação Web II

Preparando a robustez da aplicação

Programação Web II

Aumentando a Maturidade de um Projeto Web

Nesta aula, vamos explorar como aumentar a **maturidade de uma aplicação web** utilizando técnicas de validação de dados, tratamento de erros, gerenciamento de componentes e cache de dados.

Validação de Dados com Bean Validation

Programação Web II

O que é Bean Validation?

Bean Validation é uma especificação Java para validação de dados em aplicações. Ela define um conjunto de anotações para validar propriedades de classes de maneira declarativa.

Configuração no Spring Boot

Adicione a dependência
`spring-boot-starter-validation`
no arquivo `gradle.build`:

```
implementation 'org.springframework.boot:spring-boot-starter-validation'
```

Adicione a dependência
`spring-boot-starter-validation`
no arquivo `pom.xml`:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-validation</artifactId>  
</dependency>
```


Preparando a robustez da aplicação

Programação Web II

Aplicando validações

```
import jakarta.validation.constraints.*;
import lombok.Data;

@Data
public class Produto {

    @NotNull(message = "O nome não pode ser nulo")
    @Size(min = 2, max = 100, message = "O nome deve ter entre 2 e 100 caracteres")
    private String nome;

    @NotNull(message = "O preço não pode ser nulo")
    @Positive(message = "O preço deve ser positivo")
    private Double preco;
}
```

Explicação das anotações

- **@NotNull**: garante que o valor não seja null.
- **@Size**: restringe o tamanho do texto.
- **@Positive**: garante que o valor seja maior que zero.

Gerenciamento de Erros com Exception Handler

Programação Web II

Tratamento global de erros

Use a anotação `@ControllerAdvice` para criar uma classe de tratamento global de exceções.

```
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ResponseBody;
import java.util.HashMap;
import java.util.Map;

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    @ResponseBody
    public Map<String, String> handleValidationExceptions(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error ->
            errors.put(error.getField(), error.getDefaultMessage()));
        return errors;
    }
}
```

Ciclo de vida de Spring Beans

Spring Beans são componentes gerenciados pelo container do Spring. O ciclo de vida inclui:

- **Instanciação:** o Spring cria uma instância do bean.
- **Injeção de Dependências:** dependências são injetadas automaticamente.
- **Inicialização:** métodos marcados com **@PostConstruct** são executados.
- **Destruição:** métodos marcados com **@PreDestroy** são chamados antes da remoção do bean.

Gerenciamento de Componentes no Spring

Programação Web II

Definindo um bean personalizado

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class AppConfig {

    @Bean
    public String mensagemBoasVindas() {
        return "Bem-vindo ao sistema de gestão de produtos!";
    }
}
```

Explicação das anotações

- **@Configuration**: indica uma classe de configuração.
- **@Bean**: define um bean gerenciado pelo Spring.

Cache de Dados com Spring Cache

Programação Web II

Configuração Inicial

Adicione a dependência ao `pom.xml`:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-cache</artifactId>
</dependency>
```

Adicione a dependência ao `build.gradle`:

```
implementation 'org.springframework.boot:spring-boot-starter-cache'
```


Cache de Dados com Spring Cache

Programação Web II

Habilitando cache na aplicação

```
import org.springframework.cache.annotation.EnableCaching;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
@EnableCaching
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

Cache de Dados com Spring Cache

Programação Web II

Usando anotações de cache

```
import org.springframework.cache.annotation.Cacheable;
import org.springframework.cache.annotation.CacheEvict;
import org.springframework.stereotype.Service;

import java.util.Arrays;
import java.util.List;

@Service
public class ProdutoService {

    @Cacheable("produtos")
    public List<String> listarProdutos() {
        System.out.println("Consultando banco de dados ... ");
        return Arrays.asList("Notebook", "Smartphone", "Monitor");
    }

    @CacheEvict(value = "produtos", allEntries = true)
    public void limparCache() {
        System.out.println("Cache limpo!");
    }
}
```

Cuidados na manipulação de cache

Ao trabalhar com cache, é importante adotar práticas que garantam a consistência dos dados. Certifique-se de definir uma política de expurgo adequada, como o uso de estratégias **LRU** (*Least Recently Used*) ou *TTL* (*Time-To-Live*), para evitar o acúmulo excessivo de dados no cache.

Além disso, sempre invalide entradas de cache quando houver alterações nos dados subjacentes, para que os dados armazenados estejam sempre atualizados e reflitam o estado real da aplicação.

Essas práticas ajudam a evitar problemas como inconsistências e consumo desnecessário de recursos.

Exercícios

Exercícios

Preparando a robustez da aplicação

Exercício 1: Construir uma API REST em Spring Boot seguindo MVC, que consome a PokeAPI v2 e persiste/cacheia dados no H2 - Complemento

Objetivo

Revise o Projeto Api Pokémon e inclui as seguintes melhorias:

- Incluir **@Cachable** & **@CacheEvict** no métodos que fizer sentido para melhorar a robustez do projeto;
- Incluir validações nos contratos **@NotNull** e **@Size**;
- Criar um **bean personalizado** para sua aplicação.

Obrigada